

Lambda expressions and Functions as arguments to functions

(and Dictionary comprehensions)

CC-BY: Maria Skeppstedt

1. Dictionary comprehensions

A function for writing word properties in JSON format

What does this function do?

```
1 import json
2
3 def word_to_json(words: set, output_filename: str):
4     if not output_filename.endswith(".json"):
5         raise ValueError("File must end with .json")
6
7     word_info = {}
8     for word in words:
9         word_info[word] = word
10
11     with open(output_filename, "w") as write_to:
12         json.dump(word_info, write_to)
13
14 if __name__ == '__main__':
15     word_to_json({"hello", "there", "word", ","},\
16                 "plain.json")
```

A function for writing word properties in JSON format

The same with a dictionary comprehension.

```
1 import json
2
3 def word_to_json(words: set, output_filename: str):
4     if not output_filename.endswith(".json"):
5         raise ValueError("File must end with .json")
6
7     word_info = {word: word for word in words}
8
9     with open(output_filename, "w") as write_to:
10         json.dump(word_info, write_to)
11
12 if __name__ == '__main__':
13     word_to_json({"hello", "there", "word", ","},\
14                 "plain.json")
```

Not very interesting output (key is the same as value)

```
{",: ":", "hello": "hello",
"word": "word", "there": "there"}
```

2. Functions as arguments to functions

Functions as arguments

A Python function can take another function as an argument.

Such a function (i.e., the one taking another function as an argument), is one type of a **higher-order function**.

If we want to use typing, the argument is of type **Callable**.

Callable

Add a parameter, of type a Callable, that decides what information about the word to store. (Here: the `len` of the word.)

```
1 import json
2 from collections.abc import Callable
3
4 def words_to_json(words: set, output_filename: str, \
5     transform_by: Callable):
6     if not output_filename.endswith(".json"):
7         raise ValueError("File must end with .json")
8
9     word_info = {}
10    for word in words:
11        word_info[word] = transform_by(word)
12    with open(output_filename, "w") as write_to:
13        json.dump(word_info, write_to)
14
15 my_words = {"hello", "there", "word", ",", ""}
16 words_to_json(my_words, "length.json", len)
```

```
{"hello": 5, "word": 4, ",": 1, "there": 5}
```

The same with another function

Instead of calling it with `len`:

```
1 my_words = {"hello", "there", "word", ",,"}  
2 words_to_json(my_words, "length.json", len)
```

```
{"hello": 5, "there": 5, "word": 4, ",,": 1}
```

Call it with `sorted`:

```
1 words_to_json(my_words, "sorted.json", sorted)
```

```
{"hello": ["e", "h", "l", "l", "o"],  
 "there": ["e", "e", "h", "r", "t"],  
 ",,": [",,"],  
 "word": ["d", "o", "r", "w"]}
```


Send your own function

We can also define our own function, and send as the parameter `transform_by`

```
1 def words_to_json(words: set, output_filename: str, \
2     transform_by: Callable):
3     if not output_filename.endswith(".json"):
4         raise ValueError("File must end with .json")
5
6     word_info = {}
7     for word in words:
8         word_info[word] = transform_by(word)
9     with open(output_filename, "w") as write_to:
10         json.dump(word_info, write_to)
11
12 def surround_with_stars(word: str):
13     return f"*{word}*"
14
15 my_words = {"hello", "there", "word", ",",""}
16 words_to_json(my_words, "stars.json", surround_with_stars)
```

```
"hello": "*hello*", "there": "*there*", "word": "*word*", ",": "**,"
```

3. Lambda functions

Use a lambda function

By using a lambda function, you don't have to define a function with a name. Instead the function can be defined “on the fly”.

```
1 def words_to_json(words: set, output_filename: str, \
2   transform_by: Callable):
3     if not output_filename.endswith(".json"):
4         raise ValueError("File must end with .json")
5
6     word_info = {}
7     for word in words:
8         word_info[word] = transform_by(word)
9     with open(output_filename, "w") as write_to:
10         json.dump(word_info, write_to)
11
12 my_words = {"hello", "there", "word", ",", ""}
13
14 words_to_json(my_words, "stars2.json", \
15     lambda word: f"*{word}*")
```

```
"hello": "*hello*", "there": "*there*", "word": "*word*", ",": "*", ""
```

Use a lambda function

Instead of a named function:

```
1 def surround_with_stars(word):  
2     return f"*{word}*"   
3  
4 words_to_json(my_words, "stars.json", surround_with_stars)
```

You define the function when using it, without a name:

```
1 words_to_json(my_words, "stars2.json", \  
2     lambda word: f"*{word}*")
```

Use a lambda function

~~`def surround_with_stars(word):`~~
 ~~`return f" *{word} * "`~~



`lambda word: f" *{word} * "`

More lambda examples

```
1 def words_to_json(words: set, output_filename: str, \
2     transform_by: Callable):
3     if not output_filename.endswith(".json"):
4         raise ValueError("File must end with .json")
5
6     word_info = {}
7     for word in words:
8         word_info[word] = transform_by(word)
9     with open(output_filename, "w") as write_to:
10         json.dump(word_info, write_to)
11
12 my_words = {"hello", "there", "word", ",", ""}
13
14 words_to_json(my_words, "double.json", \
15     lambda word: word+word)
16 words_to_json(my_words, "upper.json", \
17     lambda word: word.upper())
18 words_to_json(my_words, "plain.json", \
19     lambda word: word)
```

More lambda examples

```
1 my_words = {"hello", "there", "word", ",","}
2
3 words_to_json(my_words, "double.json", \
4     lambda word: word+word)
5 words_to_json(my_words, "upper.json", \
6     lambda word: word.upper())
7 words_to_json(my_words, "plain.json", \
8     lambda word: word)
```

```
{"there": "therethere",
"hello": "hellohello", "word": "wordword", ",": ",,"}
```

```
{"there": "THERE", "hello": "HELLO",
"word": "WORD", ",": ",,"}
```

```
{"there": "there", "hello": "hello",
"word": "word", ",": ",,"}
```

Key for sorting

The parameter to add the function for determine how to sort is often called `key`. You can either define a named function, or using a lambda function.

```
1 def get_last_letter(word):
2     return word[-1]
3
4 my_words = ["hello", "there", "word", ",", "hello"]
5
6 print(sorted(my_words, key=len))
7 print(sorted(my_words, key=get_last_letter))
8 print(sorted(my_words, key=lambda word: word[-1]))
```

```
['', 'word', 'hello', 'there', 'hello']
['', 'word', 'there', 'hello', 'hello']
['', 'word', 'there', 'hello', 'hello']
```